



算法设计与分析

讲授内容：线性时间排序

教师：卜晨阳

联系方式：chenyangbu@hfut.edu.cn

2025年11月12日

- 课程内容：
 - 基于比较排序方法的时间复杂度下界分析
 - 计数排序
 - 基数排序

问题引入:

至今为止, 我们见过的排序都是 **比较排序**:
仅仅使用比较来比较各项的相对顺序

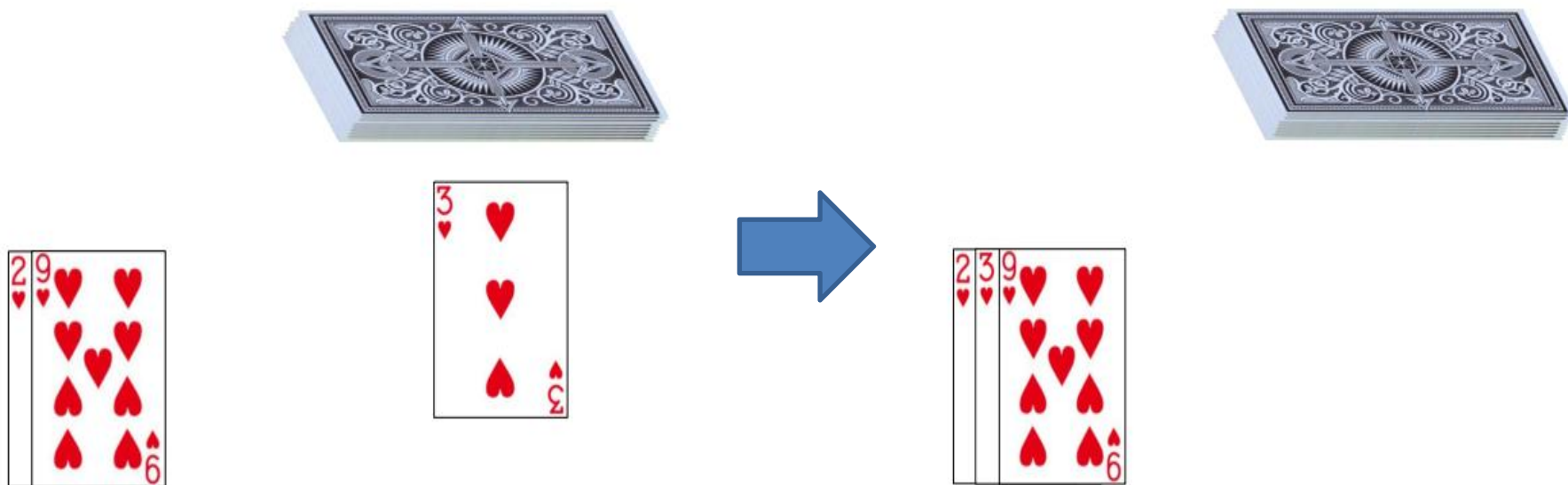
算法名称	最佳时间复杂度	平均时间复杂度	最差时间复杂度
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n^2)$

我们见过的比较排序的最好的最坏运行时间是 $O(n \lg n)$.

问题: $O(n \lg n)$ 是不是我们能做到的极限?

解决思路：以直接插入排序为例

思路：先分析一个具体的小规模例子
直接插入排序，待排序数组的规模为3



直接插入排序的核心思想举例

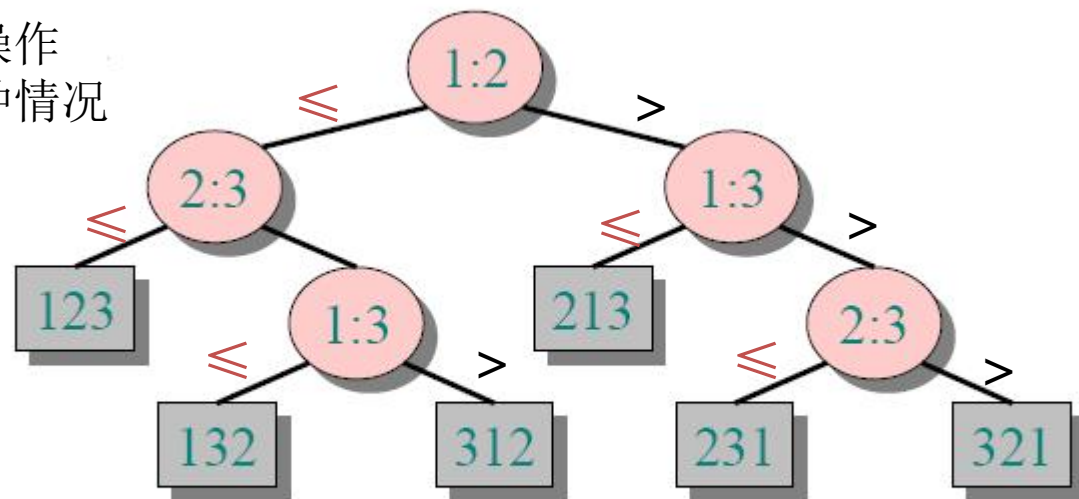
解决思路：以直接插入排序为例

待排序数组 $\langle a_1, a_2, a_3 \rangle$

每个内部节点标识为 $i:j$ $i, j \in \{1, 2, \dots, n\}$.

- 左子树表示当 $a_i \leq a_j$ 时的比较序列.
- 右子树表示当 $a_i \geq a_j$ 时的比较序列.

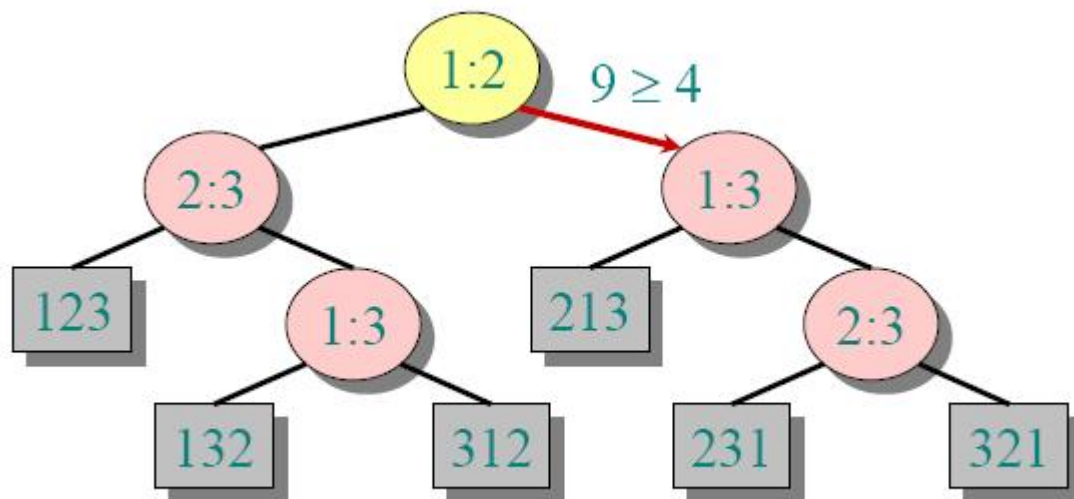
- 内部节点：对应一个比较操作
- 分支：对应一次比较的一种情况
- 叶子节点：对应一个排列



作用于3个元素的插入排序的决策树

解决思路：以直接插入排序为例

排序 $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:

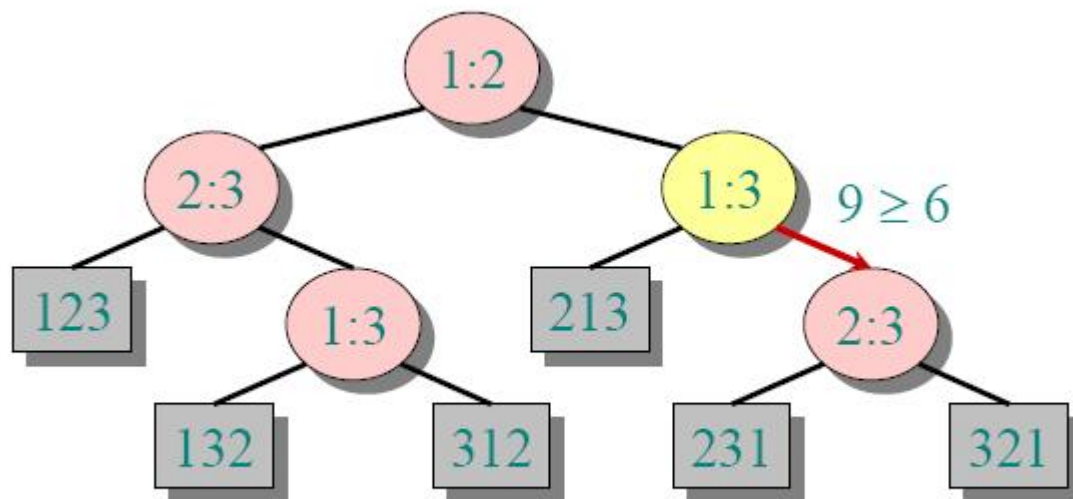


每个内部节点标识为 $i:j, i, j \in \{1, 2, \dots, n\}$.

- 左子树表示当 $a_i \leq a_j$ 时的比较序列.
- 右子树表示当 $a_i \geq a_j$ 时的比较序列.

解决思路：以直接插入排序为例

排序 $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:

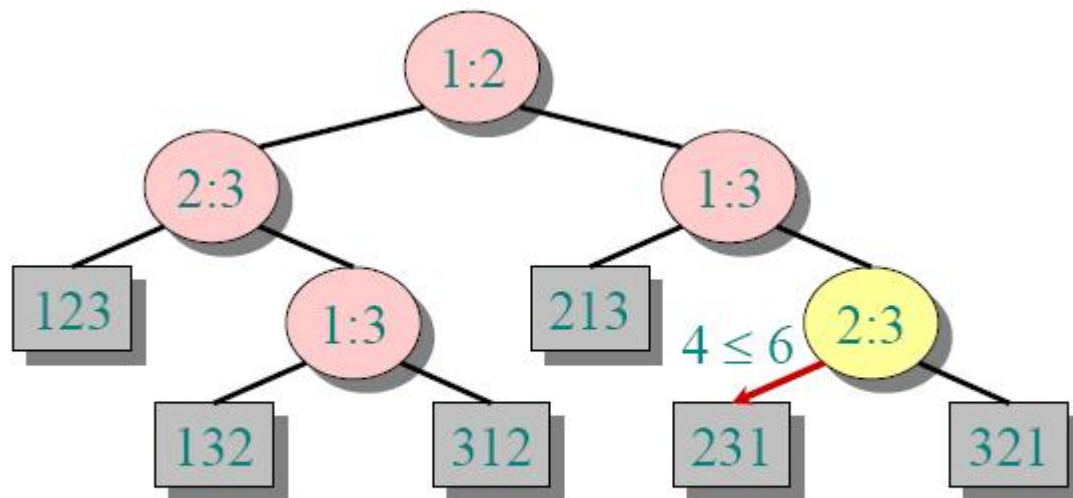


每个内部节点标识为 $i:j, i, j \in \{1, 2, \dots, n\}$.

- 左子树表示当 $a_i \leq a_j$ 时的比较序列.
- 右子树表示当 $a_i \geq a_j$ 时的比较序列.

解决思路：以直接插入排序为例

排序 $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:

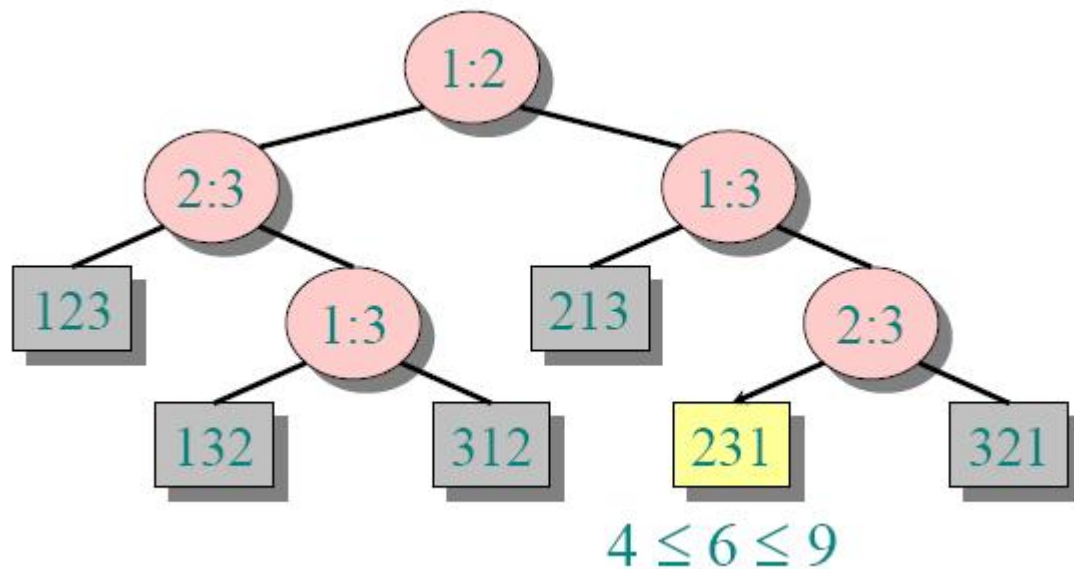


每个内部节点标识为 $i:j, i, j \in \{1, 2, \dots, n\}$.

- 左子树表示当 $a_i \leq a_j$ 时的比较序列.
- 右子树表示当 $a_i \geq a_j$ 时的比较序列.

决策树举例

排序 $\langle a_1, a_2, a_3 \rangle$
 $= \langle 9, 4, 6 \rangle$:



每个内部节点标识为 $i:j, i, j \in \{1, 2, \dots, n\}$.

- 左子树表示当 $a_i \leq a_j$ 时的比较序列.
- 右子树表示当 $a_i \geq a_j$ 时的比较序列.

归纳的结论：决策树模型

决策树可以模拟任何的比较排序的执行过程:

- 每个输入大小为 n 的序列都可以用这棵树表示.
- 将算法视为每次两个项相比后就分叉.
- 树包含了所有可能比较的指令路径.
- 算法的运行时间 = 路径的长度.
- 最坏运行时间 = 树的高度.

决策树排序的下界

定理.对 n 个项排序的任何决策树的高度是 $\Omega(n \lg n)$.

证明. 用 L 表示树的叶子节点数

$L \geq n!$ 因为 n 个元素有 $n!$ 种可能的排列.

$L \leq 2^h$ 高度 h 的二叉树有 $\leq 2^h$ 个叶子.

因此, $n! \leq 2^h$.

$$\therefore h \geq \lg(n!)$$

$$\geq \lg ((n/e)^n)$$

$$= n \lg n - n \lg e$$

$$= \Omega(n \lg n). \quad \square$$

($\lg$ 单调递增)

(Stirling's 公式) $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

推论.在最差情况下，任何一种比较排序至少需要 $O(n \lg n)$ 比较操作。这是比较操作**所获的信息**有限所导致的。从这个意义上讲，堆排序和合并排序是渐进最佳的比较排序算法。 

利用决策树证明比较排序的下界

1. 任何基于比较的排序都可以用决策树T建模
2. 对于一个特定的算法 $A \Rightarrow$ 一棵特定的树T
 1. 每一个特定的输入 $\Rightarrow$ 特定树T上的一条路径
 2. $W(n)$ 即对应树的高度
3. 比较排序下界即是所有树的最小高度
4. 而一个正确的算法，其叶子节点的个数设为L，则L满足： $n! \leq L \leq 2^h \Rightarrow h \geq \log n! \in \Omega(n \log n)$
5. 则比较排序的下界即 $n \log n$

引申（课后思考）

螺丝螺母问题的时间复杂度下界：

有一堆螺丝和一堆螺母，用肉眼无法辨别大小，螺丝只能靠螺母测大小，螺母亦然，请将它们一一配对

此题可以转化为螺丝的排序问题，但要注意的是，此题决策树的状态发生了改变，不再是二叉树，而是三叉树，多了一个“=”的decision

1. 由于将螺丝排好序后，螺母也将随之有序，所以将问题转化为螺丝排序问题
2. 使用决策树对该问题建模，定义关键操作为‘对特定螺母，螺丝比他大还是小或者正好匹配？’
3. 由于三叉树的特性，我们有 $L \leq 3^h$
4. 而输出的排列组合为 $n!$ ，于是我们有 $n! \leq L \leq 3^h \Rightarrow h \geq \log n! \Rightarrow h \in \Omega(n \log n)$
5. 所以该问题下界为 $n \log n$

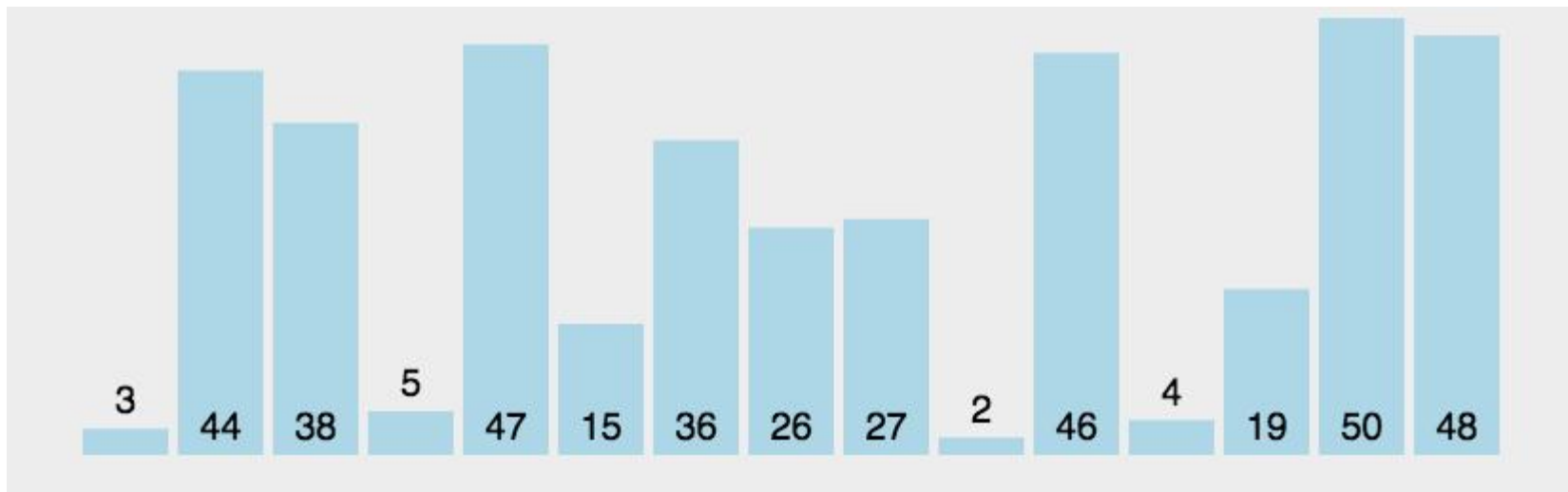
引申：简单排序算法的时间复杂度下界？

- 简单的排序算法（如冒泡、直接插入、选择排序）的最坏时间复杂度是 $O(N^2)$
- 作为直接插入排序算法的改进版本，希尔排序能够突破 $O(N^2)$ 的界，为什么？

问题：一些简单排序算法的下界？

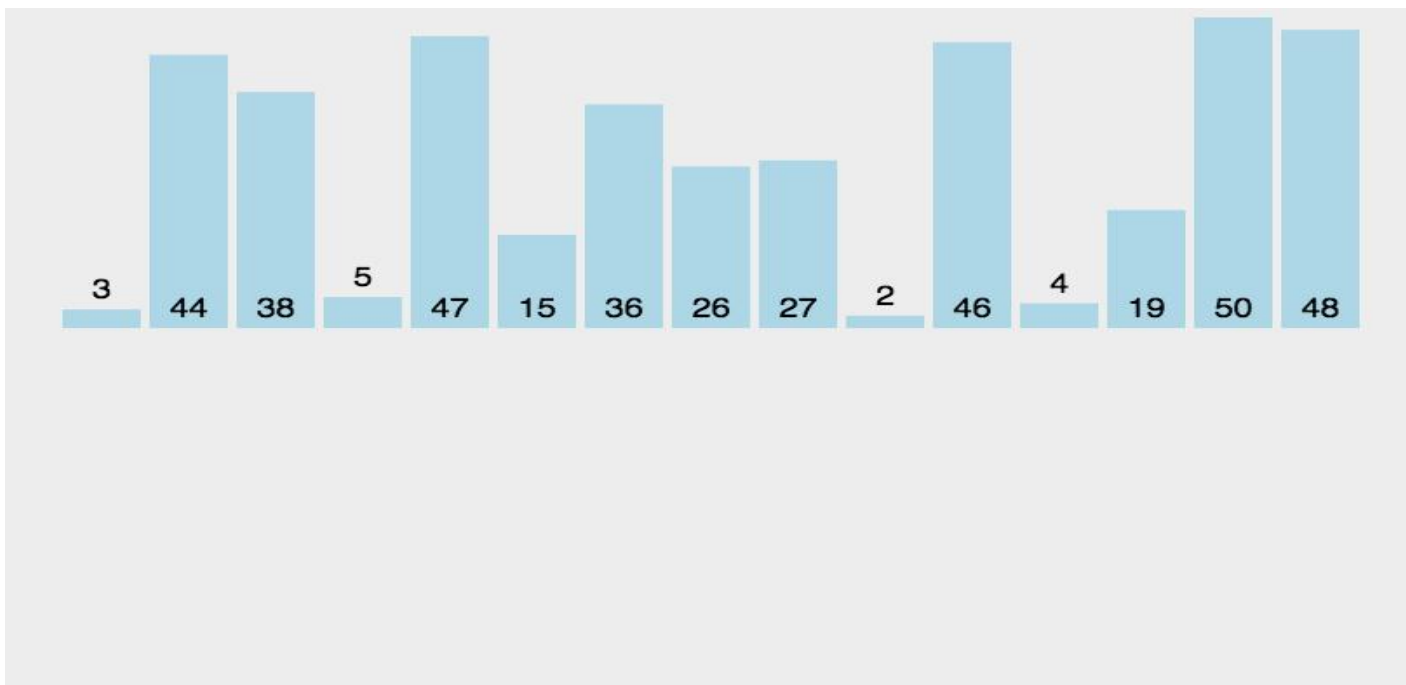
排序算法回顾：冒泡排序

- **比较相邻的元素**。如果第一个比第二个大，就交换他们两个。
- 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大的数。
- 针对所有的元素重复以上的步骤，除了最后一个。
- 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较。



排序算法回顾：插入排序

- 通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。需要反复把已排序元素逐步向后挪位，为最新元素提供插入空间。



- 定理：通过交换相邻元素进行排序的任何算法平均需要 $\Omega(N^2)$ 时间
 - 概念：逆序数
 - 排序的本质就是消除逆序数。对于随机数组，逆序数是 $O(N^2)$ 的。
 - 如果采用“交换相邻元素”的办法来消除逆序，每次正好只消除一个，因此必须执行 $O(N^2)$ 的交换次数，这就是冒泡、插入等算法只能到平方级别的原因

排序算法回顾：希尔排序（缩小增量排序）



- 是直接插入排序的改进版本。把记录按下标的一定增量分组，对每组使用直接插入排序算法排序；随着增量逐渐减少，每组包含的关键词越来越多，当增量减至1时，整个文件恰被分成一组，算法便终止。
- 问题：与直接插入相比，时间复杂度为什么可以少于 $O(n^2)$ ？

希尔能突破 $O(N^2)$ 的界，可以用逆序数来理解

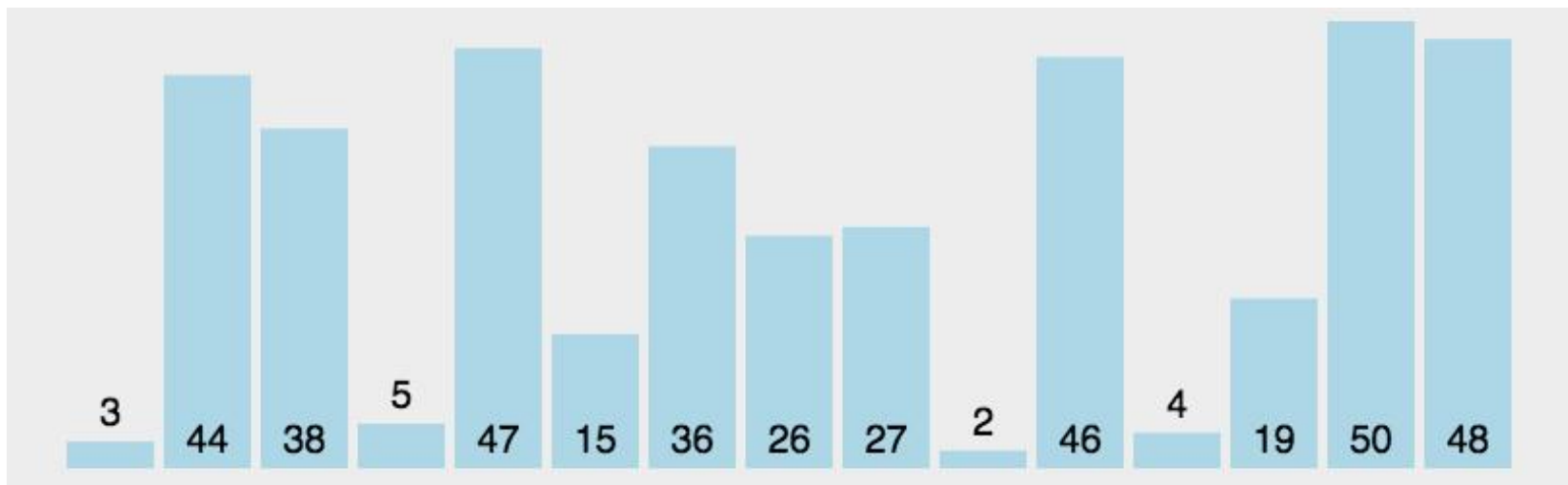
交换相隔比较远的元素，使得一次交换能消除一个以上的逆序

希尔、快排、堆排等算法都是交换比较远的元素

排序算法回顾：选择排序

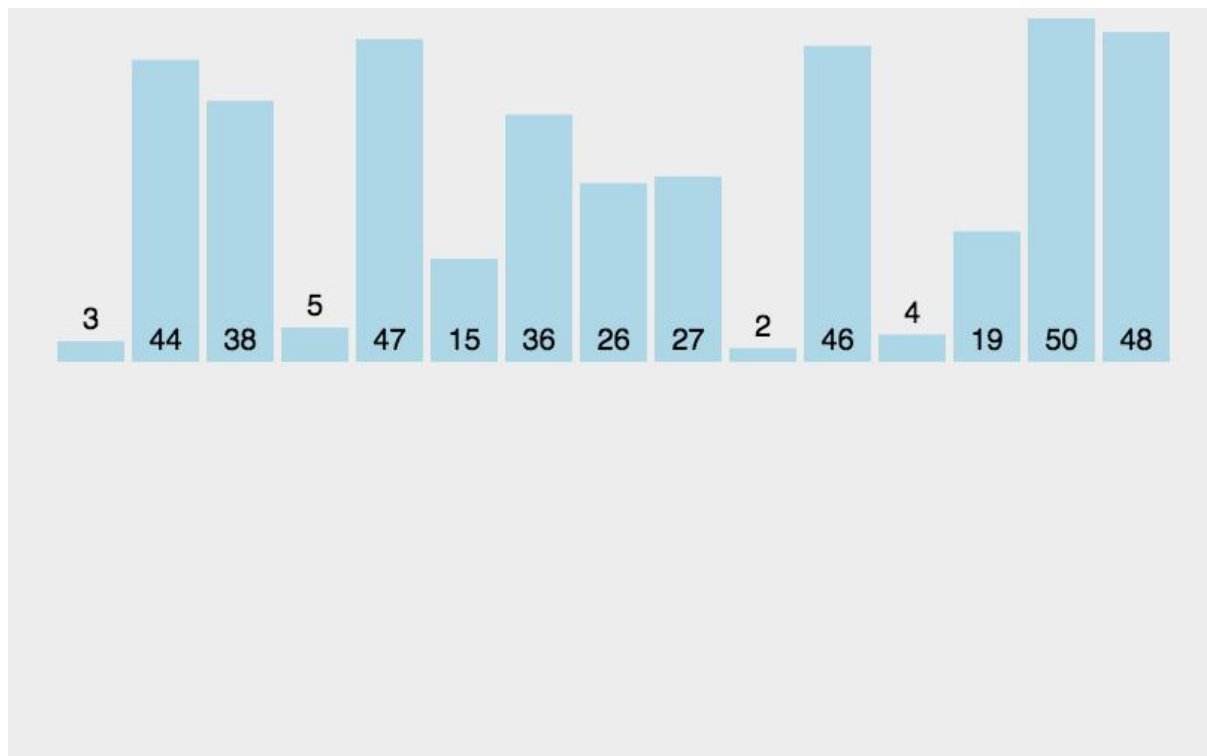
- 首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推。

问题：是否所有排序算法的最好时间复杂度都是 $O(n)$ ？



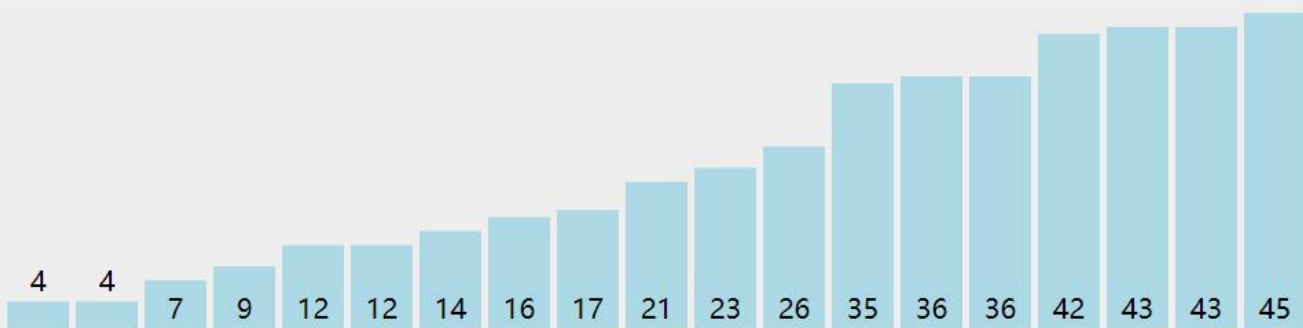
排序算法回顾：归并排序

分治法的基本思想：将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序



快速排序

基本思想：分治，通过一趟排序将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。



排序完成!

每个 (未排序) 的部分

将第一个元素设为轴心点

存储指数 = 轴心点指数 + 1

从 i=轴心点指数 + 1 到 最右指数 的遍历

如果 元素[i] < 元素[轴心点]

交换 (i, 存储指数); 存储指数++

交换 (轴心点, 存储指数 - 1)

- 求一个数组里面最小的 k 个数？

堆排序

- 堆排序：利用堆这种数据结构所设计的一种排序算法。
- 堆是一个近似完全二叉树的结构，并同时满足如下性质：即子结点的键值或索引总是小于（或者大于）它的父节点。



计数排序: 各项之间不进行比较.

- **输入:** $A[1 \dots n]$, $A[j] \in \{1, 2, \dots, k\}$.
- **输出:** $B[1 \dots n]$, 有序.
- **辅助存储:** $C[1 \dots k]$.

线性时间排序：计数排序

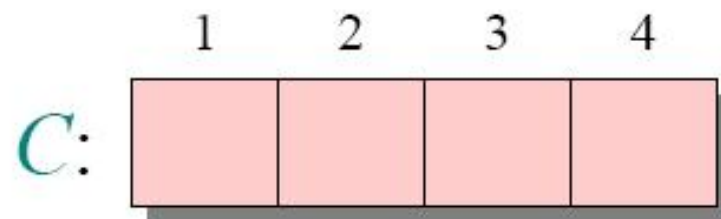
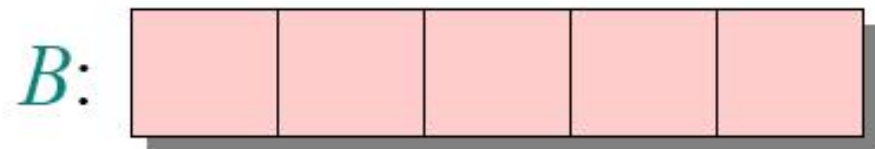
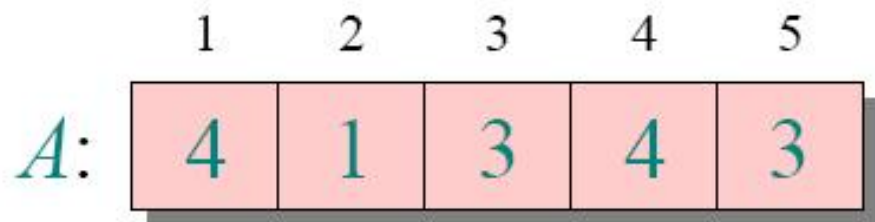
- 1954年由 Harold H. Seward 提出
- 基本思想：
 - ✓ 假设：计数排序假设 n 个输入元素中的每一个都是介于0到 k 之间的**整数**
 - ✓ 举例：10 个年龄不同的人，统计出有 8 个人的年龄比 A 小，那 A 的年龄就排在第 9 位,用这个方法可以得到其他每个人的位置,也就排好了序
 - ✓ 年龄有重复时需要特殊处理（保证稳定性） 算法最后一步
- 优势在于在对一定范围内的整数排序时，它的复杂度为 $O(n+k)$ （其中 k 是整数的范围），快于任何比较排序算法
- 当然这是一种牺牲空间换取时间的做法，而且当 $O(k) > O(n \log(n))$ 的时候其效率反而不如基于比较的排序

线性时间排序：计数排序

- 算法基本步骤：

- (1) 找出待排序的数组中最大和最小的元素
- (2) 统计数组中每个值为 i 的元素出现的次数，存入数组C的第 i 项
- (3) 对所有的计数累加（从C中的第一个元素开始，每一项和前一项相加）
- (4) 反向填充目标数组：将每个元素 i 放在新数组的第 $C(i)$ 项，每放一个元素就将 $C(i)$ 减去1

计数排序举例



循环1

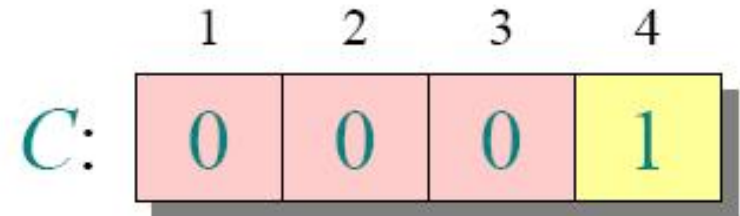
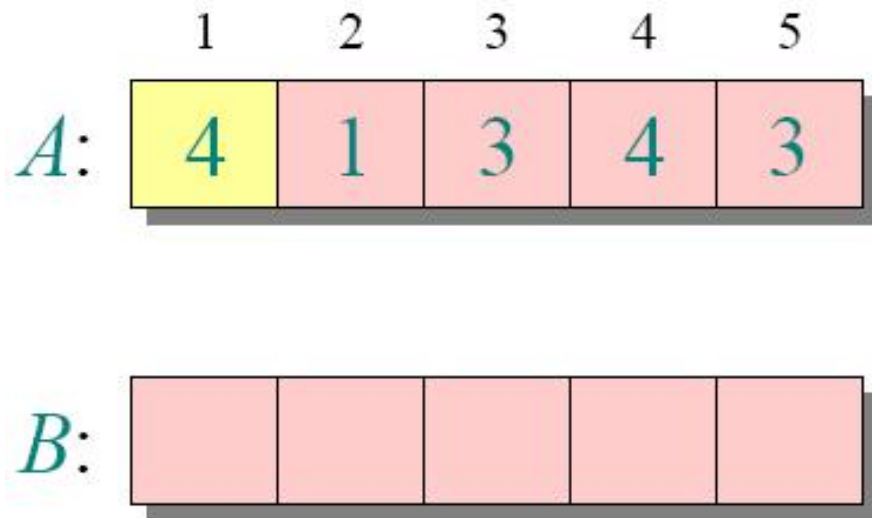
	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	0	0	0	0

B :					
-------	--	--	--	--	--

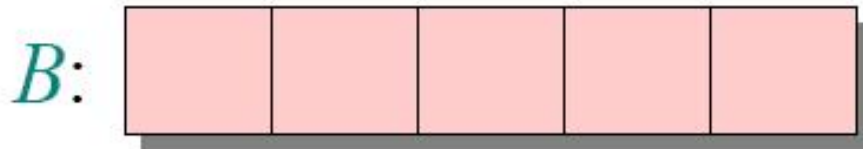
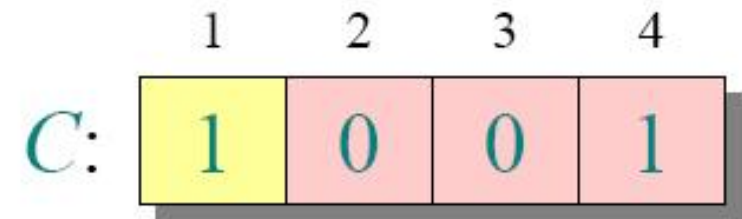
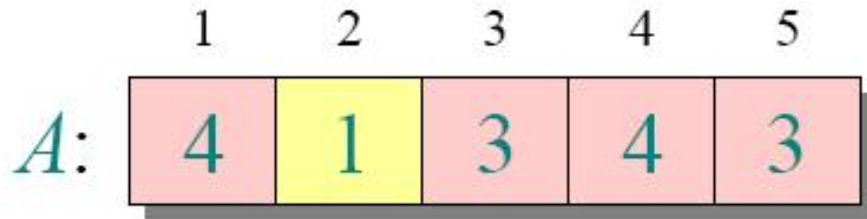
```
for  $i \leftarrow 1$  to  $k$   
  do  $C[i] \leftarrow 0$ 
```

循环2



for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

循环2



for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

循环2

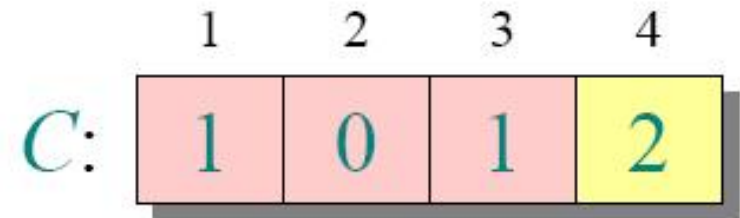
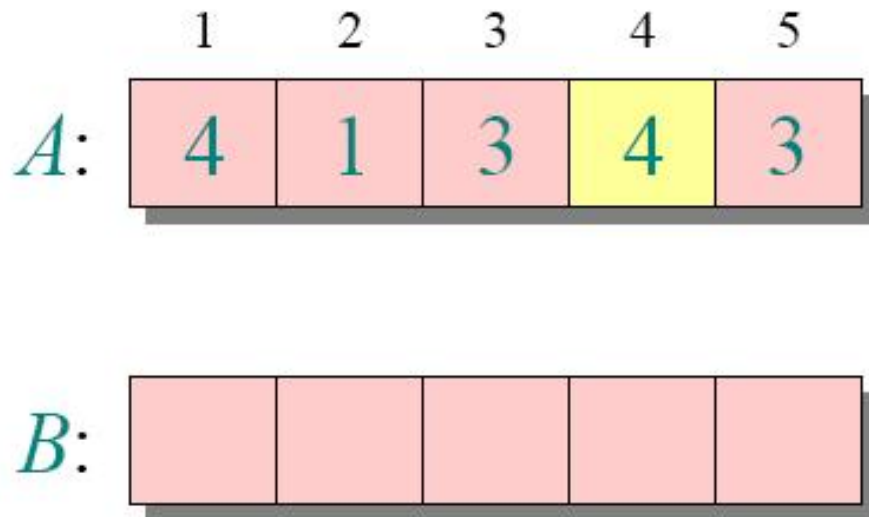
	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	1	0	1	1

B :					
-------	--	--	--	--	--

for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

循环2



for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

循环2

	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	1	0	2	2

B :					
-------	--	--	--	--	--

for $j \leftarrow 1$ **to** n
 do $C[A[j]] \leftarrow C[A[j]] + 1$ $\triangleright$ $C[i] = |\{\text{key} = i\}|$

循环3

	1	2	3	4	5
A :	4	1	3	4	3

	1	2	3	4
C :	1	0	2	2

B :					
-------	--	--	--	--	--

C' :	1	1	2	2
--------	---	---	---	---

for $i \leftarrow 2$ to k
do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

<i>B</i> :					
------------	--	--	--	--	--

	1	2	3	4
<i>C</i> :	1	0	2	2

<i>C'</i> :	1	1	3	2
-------------	---	---	---	---

for $i \leftarrow 2$ to k
do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环3

	1	2	3	4	5
<i>A</i> :	4	1	3	4	3

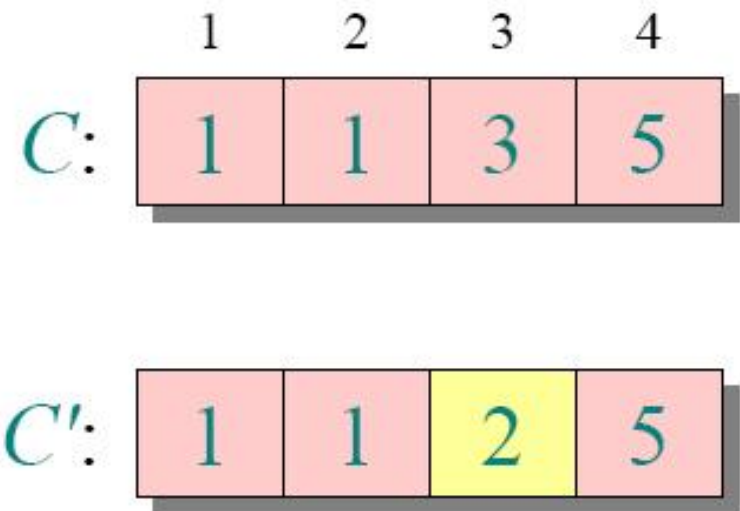
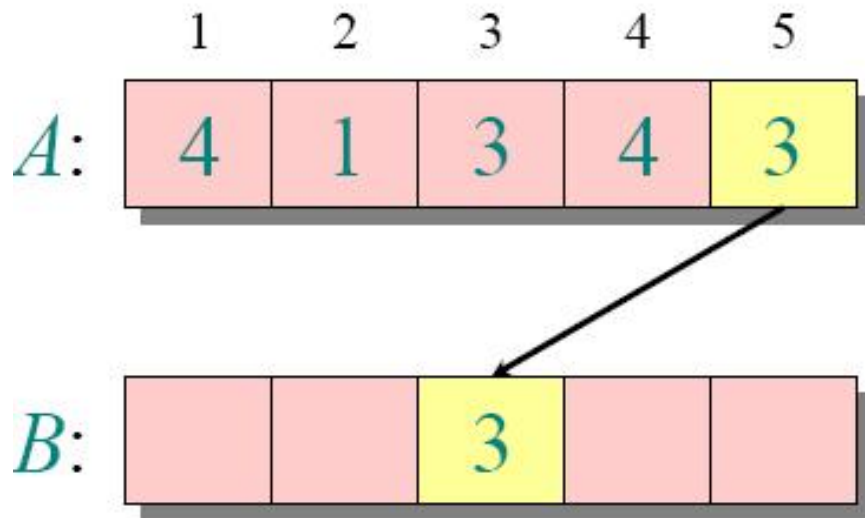
	1	2	3	4
<i>C</i> :	1	0	2	2

<i>B</i> :					
------------	--	--	--	--	--

<i>C'</i> :	1	1	3	5
-------------	---	---	---	---

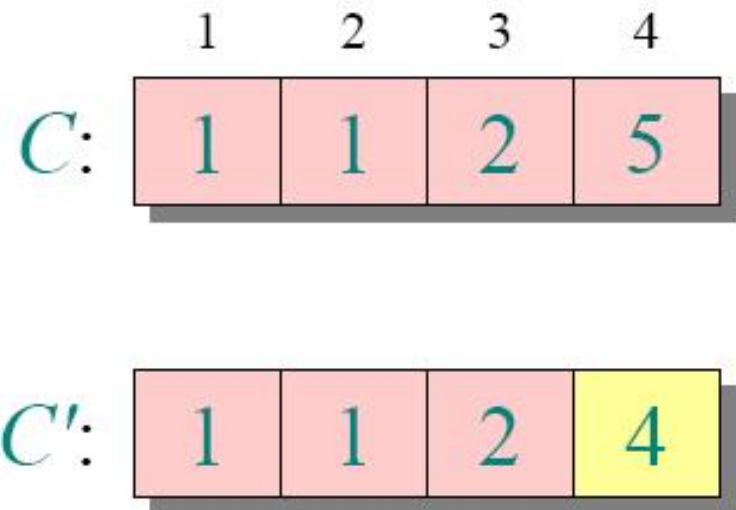
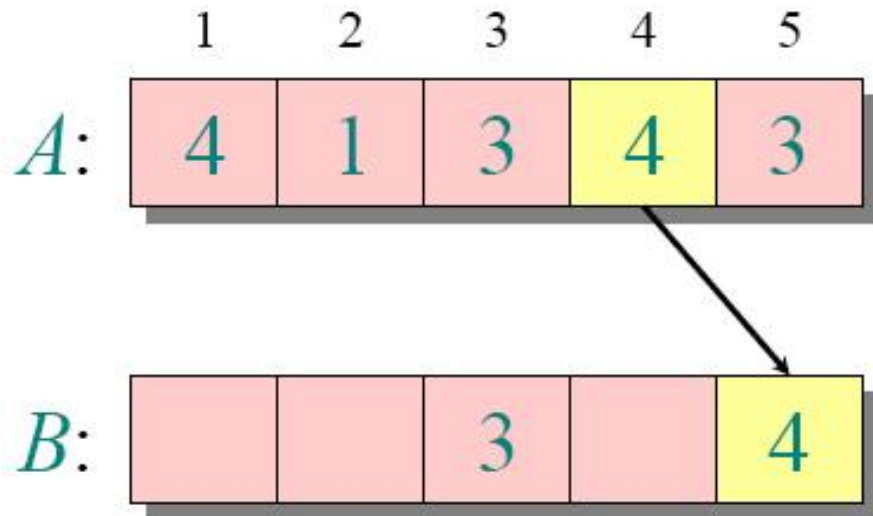
for $i \leftarrow 2$ to k
do $C[i] \leftarrow C[i] + C[i-1]$ $\triangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环4



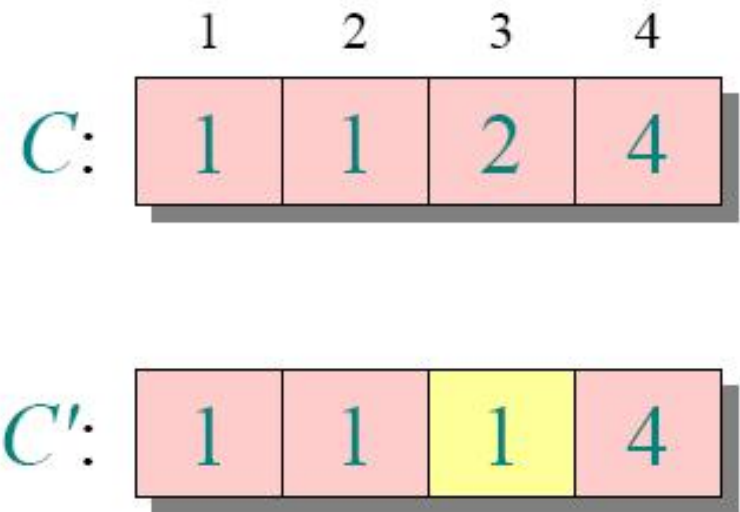
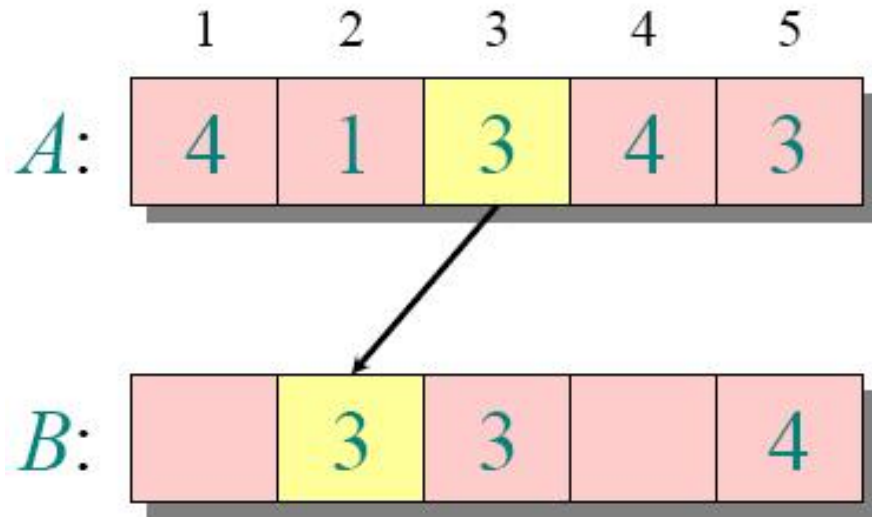
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```


循环4



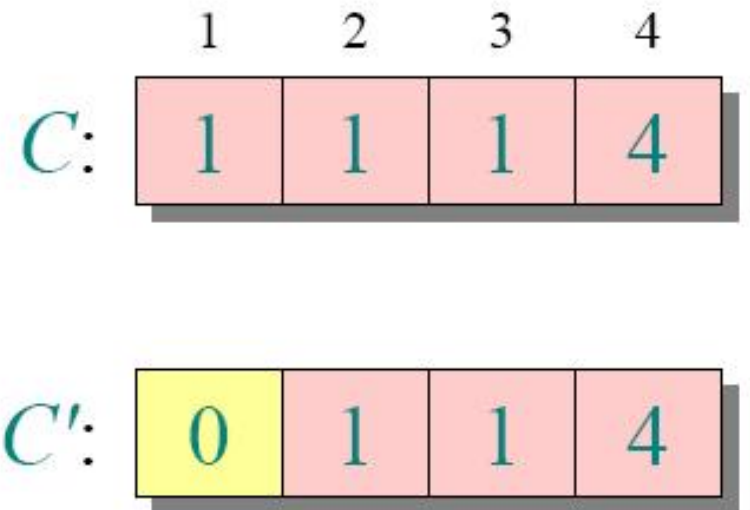
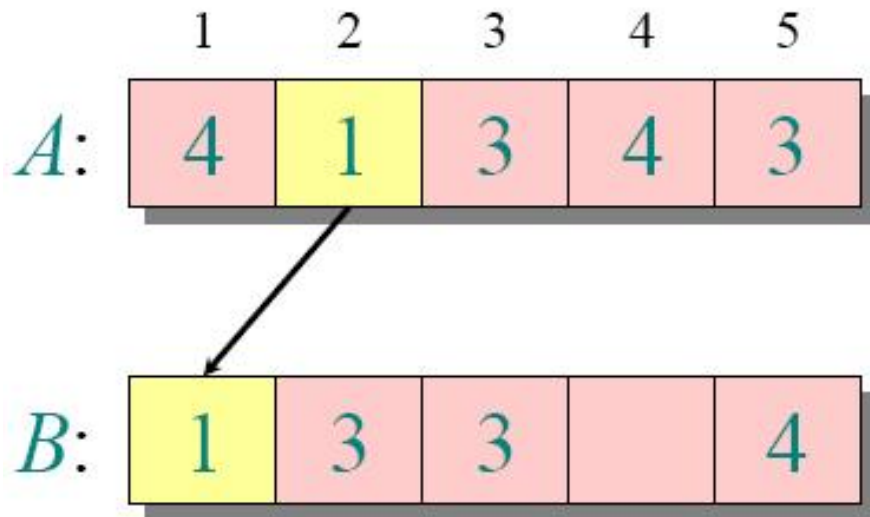
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

循环4



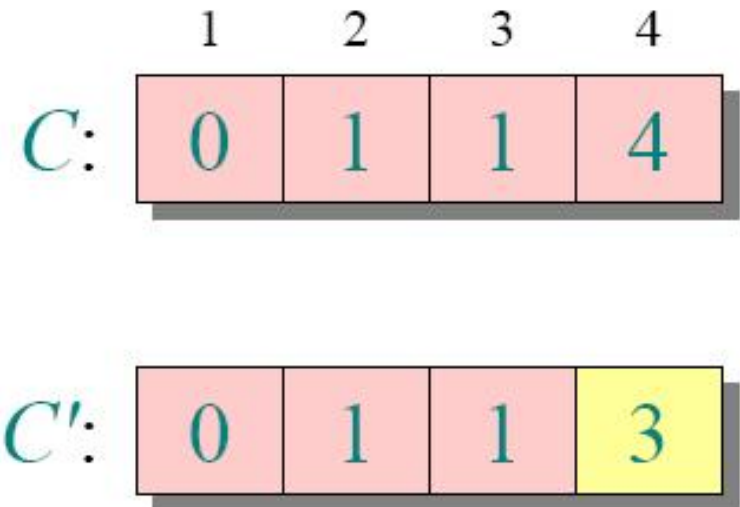
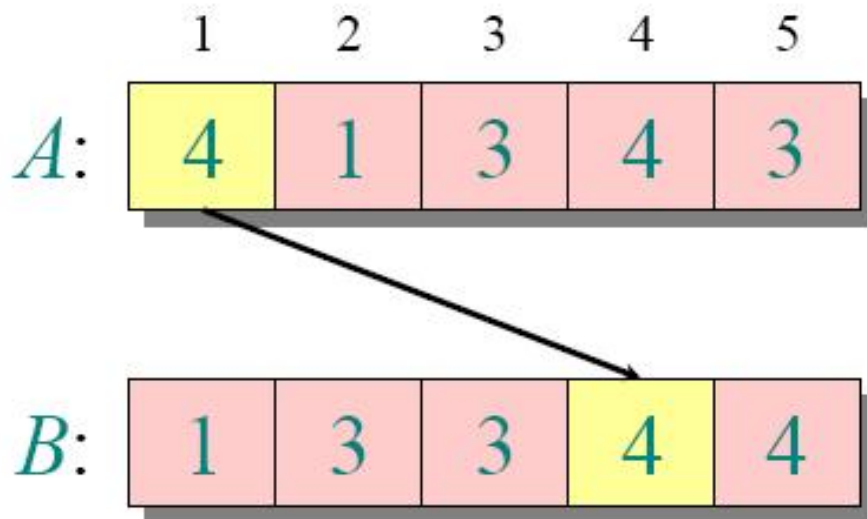
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

循环4



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

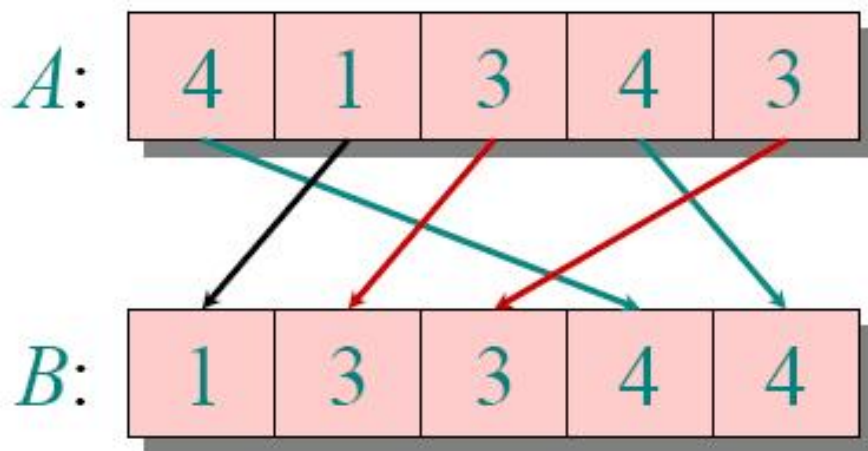
循环4



```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

```
for  $i \leftarrow 1$  to  $k$ 
    do  $C[i] \leftarrow 0$ 
for  $j \leftarrow 1$  to  $n$ 
    do  $C[A[j]] \leftarrow C[A[j]] + 1$   $\triangleright C[i] = |\{\text{key} = i\}|$ 
for  $i \leftarrow 2$  to  $k$ 
    do  $C[i] \leftarrow C[i] + C[i-1]$   $\triangleright C[i] = |\{\text{key} \leq i\}|$ 
for  $j \leftarrow n$  downto 1
    do  $B[C[A[j]]] \leftarrow A[j]$ 
        $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

计数排序是一种**稳定**排序：它会保持相等的项的相对顺序。



练习：哪种其他的排序有这种特征？

$\Theta(k)$ { for $i \leftarrow 1$ to k
do $C[i] \leftarrow 0$

$\Theta(n)$ { for $j \leftarrow 1$ to n
do $C[A[j]] \leftarrow C[A[j]] + 1$

$\Theta(k)$ { for $i \leftarrow 2$ to k
do $C[i] \leftarrow C[i] + C[i-1]$

$\Theta(n)$ { for $j \leftarrow n$ downto 1
do $B[C[A[j]]] \leftarrow A[j]$
 $C[A[j]] \leftarrow C[A[j]] - 1$

$\Theta(n + k)$

如果 $k = O(n)$, 那么计数排序用的时间为 $\Theta(n)$.

- 但是, 排序的时间是 $\Omega(n \lg n)$!

- 问题出在什么地方?

答案:

- **比较排序** 的时间是 $\Omega(n \lg n)$.

- 计数排序不是 **比较排序**.

- 实际上, 各项之间根本没有比较!

空间复杂度: $O(k)$

缺点: m 比较大时的复杂度

线性排序：基数排序

- 基本思想：将整数按位数切割成不同的数字，然后按每个位数分别比较（元素级别的分治法）
- 所谓的基数（radix），也就是“进制”的意思，所谓的“每一位”是该进制下的位
- 实现方式：
 - 1) 将所有待比较数值（正整数）统一为同样的数位长度，数位较短的数前面补零。
 - 2) 从最低位开始，依次进行一次排序。
 - 从最低位排序一直到最高位排序完成以后，数列就变成一个有序序列。

- 发源：Herman Hollerith 为 1890年美国人口普查设计的卡排序机
- 一位一位的排序.
- Hollerith 最初(不好)的想法:从最高位开始排序.
- 好的想法:使用辅助的稳定排序方法从 **最低位开始** 排序 .

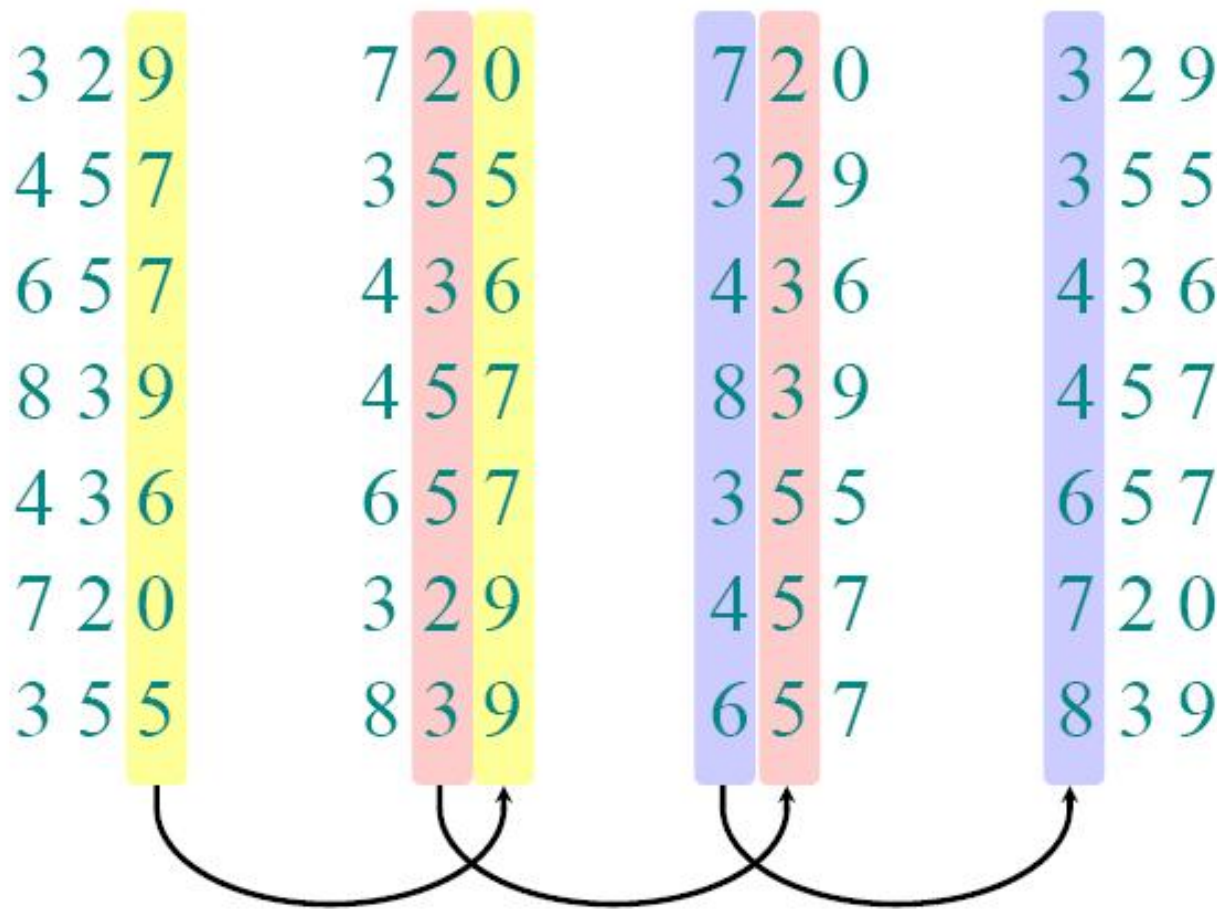
通俗地讲，之所以先排低位再排高位，是因为越是后排的数位，其对结果次序的影响越大，很显然是高位比低位对数的大小影响大！

Hollerith在1889年的最初专利展示了最高位优先的基数排序：

“The most complicated combinations can readily be counted with comparatively few counters or relays by first assorting the cards according to the first items entering into the combinations, then reassorting each group according to the second item entering into the combination, and so on, and finally counting on a few counters the last item of the combination for each group of cards.”

最低位优先的基数排序好像是由一位机器操作员发明的。

基数排序过程



RADIX-SORT(A, d)

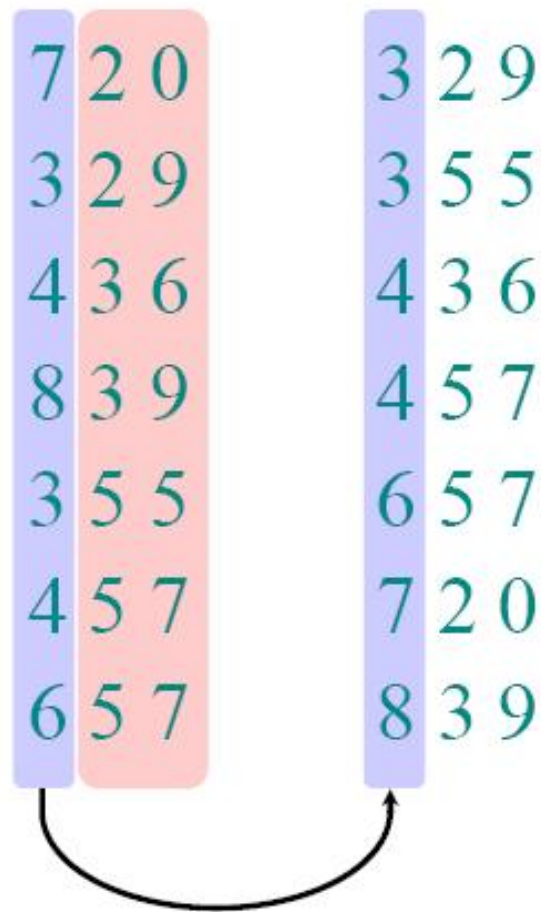
for $i \leftarrow 1$ **to** d

do use a stable sort to sort array A on digit i

基数排序的正确性

基本推导思路：

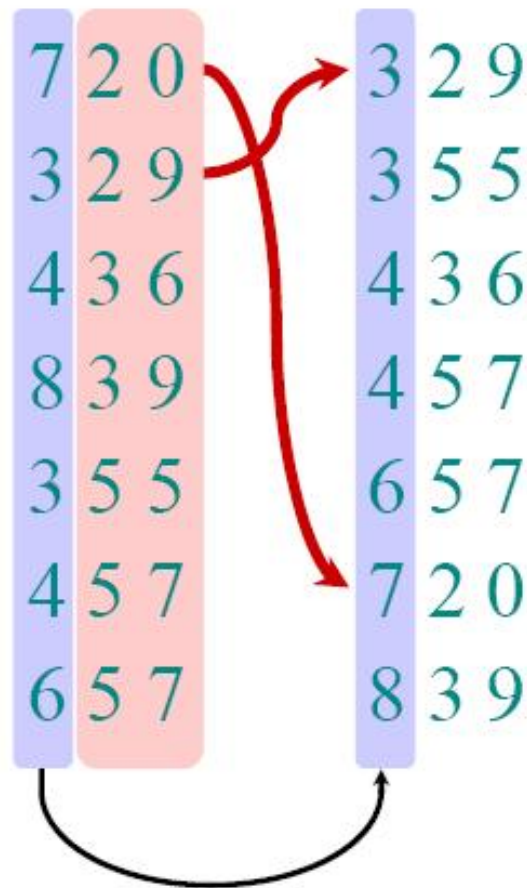
- 假定一组序列低 $t-1$ 位已经排序好了，现在我们按照第 t 位进行排序。
- 只需证明对于任意两个数来说，按照第 t 位排序之后其相对位置正确。
- 任意两个数这时有两种情况：
 - 1) 它们第 t 位相同，那么此时如果我们保持它们位置的稳定性，那么它们最后仍然有序；
 - 2) 它们第 t 位不同，按照第 t 位排序之后就有序了。



基数排序的正确性

数位推导

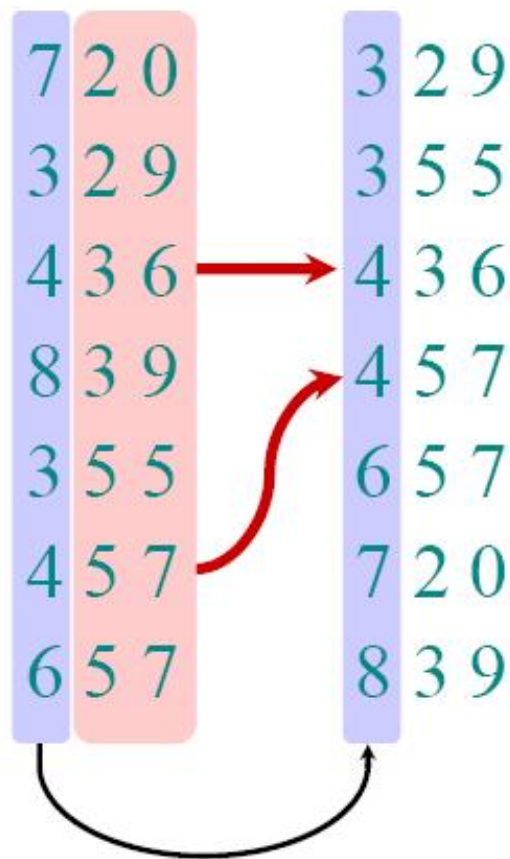
- 假设数字已经按照其低阶 $t-1$ 位排序.
- 按照 t 位排序
 - 两个在位 t 不同的数被正确排序.



基数排序的正确性

数位推导

- 假设数字已经按照其低阶 $t-1$ 位排序.
- 按照 t 位排序
 - 两个在位 t 不同的数被正确排序.
 - 两个 t 位相同的数保持与输入相同的次序 $\Rightarrow$ 正确的顺序.

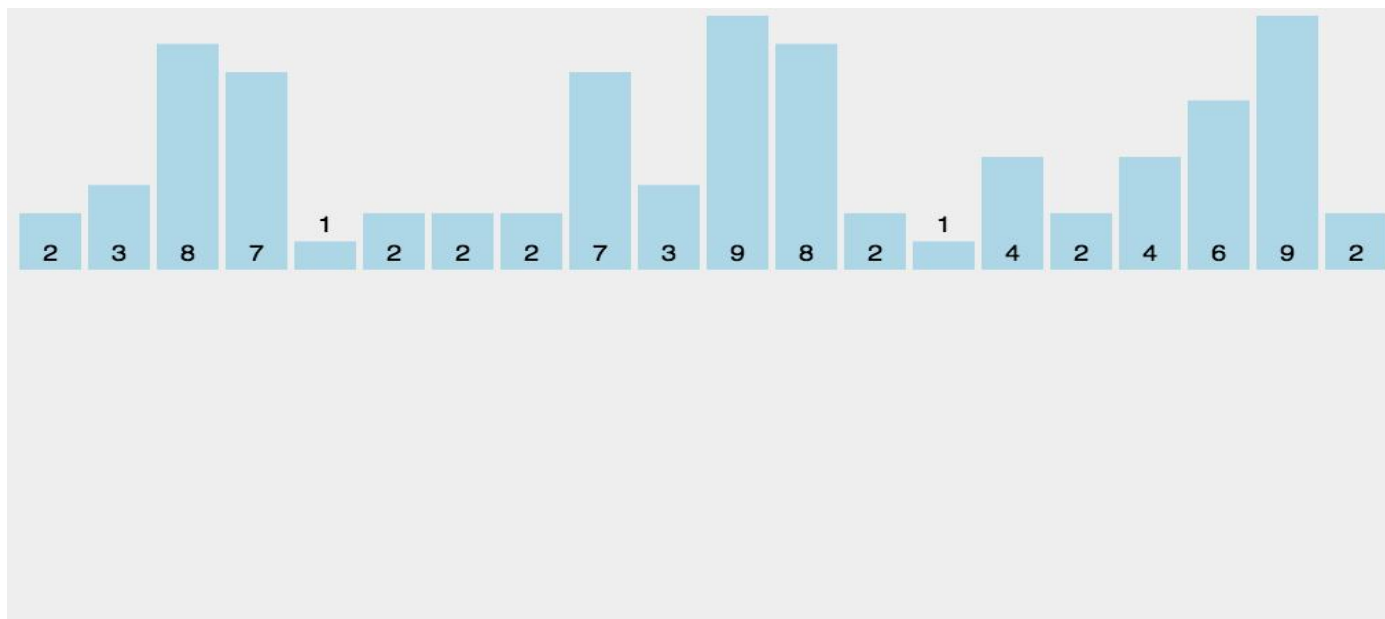


基数排序比较适合对取值很大的数进行排序，也可用来对字符串进行排序。

但基数排序的缺点是不呈现时空局部性，因为在按位对每个数进行排序的过程中，一个数的位置可能发生巨大的变化，所以不能充分利用现代机器缓存提供的优势。

同时计数排序作为中间稳定排序的话，不具有原地排序的特点，当内存容量比较宝贵的时候，还是有待商榷。

- 计数排序使用一个额外的数组C，其中第*i*个元素是待排序数组A中值等于*i*的元素的个数。然后根据数组C来将A中的元素排到正确的位置。它只能对整数进行排序。



- 基本思路：将所有待比较数值（正整数）统一为同样的数位长度，数位较短的数前面补零。然后，从最低位开始，依次进行一次排序。这样从最低位排序一直到最高位排序完成以后，数列就变成一个有序序列。

3	44	38	5	47	15	36	26	27	2	46	4	19	50	48
---	----	----	---	----	----	----	----	----	---	----	---	----	----	----

- 《数据结构》课程中已学习过的排序算法
- 基于决策树模型分析比较排序算法的时间复杂度下界
- 计数排序
- 基数排序



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队
HFUT Data Mining and Intelligent Computing Laboratory